```
separate (IDL.SEM_PHASE)¶    --|-------------------------------------------------------------------------------------------¶    --| HOM_UNIT¶    --
|-------------------------------------------------------------------------------------------¶package body HOM_UNIT is¶  use VIS_UTIL;¶  use DEF_UTI
L;¶  use RED_SUBP;¶  use SET_UTIL;¶  use EXP_TYPE, EXPRESO;¶  use ATT_WALK;¶  use MAKE_NOD;¶¶      --|||||||||||||||||||||||||||||||||||||||||||||||||
|||||||||||||||||||||||||||||||||||||||||||¶      --|¶  function WALK_HOMOGRAPH_UNIT (UNIT_NAME : TREE;¬HEADER : TREE) return TREE is¶    NEW_NAME   :
 TREE¬  := UNIT_NAME;¶    INDEX¬     : TREE¬  := TREE_VOID;¶    TYPESET    : TYPESET_TYPE¬  := EMPTY_TYPESET;¶    DEFSET     : DEFSET_TYPE;¶    DEFINT
ERP  : DEFINTERP_TYPE;¶    DEF¬      : TREE;¶    DEF_HEADER : TREE;¶    NEW_DEFSET : DEFSET_TYPE¬  := EMPTY_DEFSET;¶    INDEX_OK   : Boolean¬  := True;
¶    EXTRAINFO  : EXTRAINFO_TYPE := NULL_EXTRAINFO;¶    DUMMY_FLAG : Boolean;¶  begin¶    if NEW_NAME.TY = DN_STRING_LITERAL then¶      NEW_NAME := MA
KE_USED_OP_FROM_STRING (NEW_NAME);¶    end if;¶¶    if NEW_NAME.TY = DN_FUNCTION_CALL and then (D¬(AS_NAME,¬NEW_NAME).TY = DN_USED_OBJECT_ID or else D
 (AS_NAME, NEW_NAME).TY = DN_SELECTED) and then¬IS_EMPTY (TAIL (LIST (D (AS_GENERAL_ASSOC_S, NEW_NAME))))¶     and then HEAD (LIST (D (AS_GENERAL_ASSO
C_S, NEW_NAME))).TY /= DN_ASSOC¶    then¶      INDEX    := HEAD (LIST (D (AS_GENERAL_ASSOC_S, NEW_NAME)));¶      NEW_NAME := D¬(AS_NAME,¬NEW_NAME);¶
    EVAL_EXP_TYPES (INDEX, TYPESET);¶    end if;¶¶    if NEW_NAME.TY = DN_ATTRIBUTE then¶      EVAL_ATTRIBUTE (NEW_NAME, TYPESET, DUMMY_FLAG, IS_FUNCT
ION => True);¶¬¬    -- $$$$ SHOULD CHECK FOR VALID ATTRIBUTE¶      NEW_NAME := RESOLVE_ATTRIBUTE (NEW_NAME);¶    elsif¬NEW_NAME.TY = DN_USED_OBJECT_ID
 or else¬NEW_NAME.TY = DN_USED_OP or else NEW_NAME.TY = DN_USED_CHAR¬or else NEW_NAME.TY¬= DN_SELECTED then¶      FIND_VISIBILITY (NEW_NAME, DEFSET);¶
      if not IS_EMPTY (DEFSET) then¶        while not IS_EMPTY (DEFSET) loop¶¬POP (DEFSET, DEFINTERP);¶¬DEF¬ := GET_DEF (DEFINTERP);¶¬DEF_HEADER := D
(XD_HEADER, DEF);¶¬if INDEX = TREE_VOID then¶¬  if DEF_HEADER.TY in DN_PROCEDURE_SPEC¬.. DN_FUNCTION_SPEC¬or else (DEF_HEADER.TY = DN_ENTRY and then D
 (AS_DISCRETE_RANGE, DEF_HEADER) = TREE_VOID)¬then¶¬    if ARE_HOMOGRAPH_HEADERS (HEADER, DEF_HEADER)¬then¶¬      ADD_TO_DEFSET¬(NEW_DEFSET, DEFINTERP
);¶¬    end if;¶¬  end if;¶¬else¶¬¬¬¬         -- RETRIEVE¬THE HEADER FOR ENTRY FAMILY MEMBER¶¬  if D (XD_SOURCE_NAME, DEF).TY = DN_ENTRY_ID then¶¬    D
EF_HEADER := D¬(SM_SPEC,¬D (XD_SOURCE_NAME, DEF));¶¬  end if;¶¶¬  if DEF_HEADER.TY = DN_ENTRY¬and then D (AS_DISCRETE_RANGE, DEF_HEADER) /= TREE_VOID
then¶¬    CHECK_ACTUAL_TYPE (GET_TYPE_OF_DISCRETE_RANGE¬(D (AS_DISCRETE_RANGE, DEF_HEADER)), TYPESET, INDEX_OK, EXTRAINFO);¶¬    if INDEX_OK and¬then
IS_SAME_PARAMETER_PROFILE (D (AS_PARAM_S, HEADER), D (AS_PARAM_S, DEF_HEADER)) then¶¬      ADD_EXTRAINFO¬(DEFINTERP, EXTRAINFO);¶¬      ADD_TO_DEFSET¬
(NEW_DEFSET, DEFINTERP);¶¬    end if;¶¬  end if;¶¬end if;¶        end loop;¶        DEFSET := NEW_DEFSET;¶        if IS_EMPTY¬(DEFSET) then¶¬if NEW_NA
ME.TY = DN_SELECTED then¶¬  ERROR (D (LX_SRCPOS, UNIT_NAME), "NO MATCHING SUBPROGRAMS - " & PRINT_NAME (D¬(LX_SYMREP, D (AS_DESIGNATOR,¬NEW_NAME))));¶
¬else¶¬  ERROR (D (LX_SRCPOS, UNIT_NAME), "NO MATCHING SUBPROGRAMS - " & PRINT_NAME (D¬(LX_SYMREP, NEW_NAME)));¶¬end if;¶        end if;¶        REQUI
RE_UNIQUE_DEF (NEW_NAME, DEFSET);¶        NEW_NAME :=¬RESOLVE_NAME (NEW_NAME, GET_THE_ID (DEFSET));¶        if INDEX /=¬TREE_VOID¬then¶¬if IS_EMPTY (D
EFSET) then¶¬  INDEX := RESOLVE_EXP (INDEX, TREE_VOID);¶¬else¶¬  INDEX := RESOLVE_EXP (INDEX, GET_TYPE_OF_DISCRETE_RANGE (D (AS_DISCRETE_RANGE, DEF_HE
ADER)));¶¬end if;¶¬NEW_NAME := MAKE_INDEXED (AS_NAME => NEW_NAME, AS_EXP_S => MAKE_EXP_S¬(LIST => SINGLETON (INDEX), LX_SRCPOS => D (LX_SRCPOS, D (AS_
GENERAL_ASSOC_S, UNIT_NAME))), LX_SRCPOS => D (LX_SRCPOS, UNIT_NAME));¶        end if;¶      end¬if;¶    else¶      ERROR (D (LX_SRCPOS, NEW_NAME), "C
ANNOT BE SUBPROGRAM NAME");¶      NEW_NAME := RESOLVE_EXP¬(NEW_NAME, TREE_VOID);¶    end if;¶¶    return NEW_NAME;¶  end WALK_HOMOGRAPH_UNIT;¶¶   --|-
-------------------------------------------------------------------------------------------¶end HOM_UNIT;¶
```